Software Engineering

Lab 4

Vedant Sharma

PES2UG24CS577

Section 5-J

https://github.com/QVedant/SE_Scenario17_Wordle

https://github.com/QVedant/Software-Engineering-Lab

Files uploaded to github.

Main Bug Prefix

```
Wordle – 5 letters, 6 guesses.
> apple
green gray gray gray gray
> arayy
green green yellow yellow green
> array
green green green green green
Solved!
```

Here “array” is the answer, so the guess “arayy” should have only last “y” as green, and the unnecessary “y” should not be coloured.

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab4\17_wordle> python main.py
Wordle – 5 letters, 6 guesses.
> appel
green green green yellow yellow
> allay
green yellow yellow yellow gray
> apple
green green green green green
Solved!
```

Similarly, here “allay” has double “l”, only one of which should have been coloured, yet both are marked yellow.

Task 1 Fix

```
Wordle – 5 letters, 6 guesses.
> appel
yellow yellow gray yellow gray
> graep
green green green yellow yellow
> graap
green green green gray yellow
> ggrape
Enter a valid word of the required length.
> ggrap
green gray yellow yellow yellow
> grape
green green green green green
Solved!
```

Fixed the double/multi occurrence colouring, and the perfect fit match colour hierarchy.

Task 2 Fix

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab4\17_wordle> python main.py
Wordle – 5 letters, 6 guesses.
> one
Enter a valid word of the required length.
> gues1
Enter a valid word of the required length.
> guess
gray gray gray gray gray
> secon
gray gray gray gray gray
> third
gray gray gray yellow gray
> fourt
gray gray gray yellow gray
> fifth
gray gray gray gray gray
> sixt
Enter a valid word of the required length.
> sixth
gray gray gray gray gray
Out of guesses!
The word was: array
```

Invalid guess doesn't count for the 6 chances per round. Only 6 valid are counted. It detects incorrect word length.

Task 3 Fix

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab4\17_wordle> python main.py
Choose word length (4, 5, or 6): 4
Wordle — 4 letters, 6 guesses.
> apple
Enter a valid word of the required length.
> appl
gray gray gray yellow
> lima
yellow gray gray gray
> ploy
gray green gray gray
> blay
green green gray gray
> blue
green green green green
Solved!
```

4 and 6 length words are a guess option now. Can be set in words.py

Task 4 Fix

```
Solved!

--- Session Summary ---
Outcome: Won
Target word: crane
Valid guesses: 5/6
Total guesses:
1. apple - yellow gray gray gray green
2. falme - gray yellow gray gray green
3. staer - gray gray green yellow yellow
4. grape - gray green green gray green
5. crane - green green green green green
----------------------
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab4\17_wordle>
```

Each guess is saved in a history, with its resulting colour sequence. This helps the user with keeping track.

Chat History

Project materials

17_wordle-main.zip — Wordle game starter project.

Lab_4_VibeCoding_Student_handout.pdf — lab instructions and task requirements.

Gameplay screenshot — used to document the observed bug.

Chronological prompt history

1. Review the project

User prompt: "Please inspect the Wordle project and the Lab 4 handout first. Understand the current code structure and the requirements before making changes."

Purpose: Establish the starter code and lab handout as the reference for the work.

2. Document the gameplay bug

User prompt: "I captured a screenshot of the bug I noticed while playing. Please use it to understand the issue and keep the fix aligned with the existing game."

Purpose: Connect the observed gameplay behavior to the code that needs attention.

3. Task 1 — duplicate-letter feedback

User prompt: "Let's fix Task 1 in feedback.py. Correct the handling of repeated letters while preserving the existing evaluate function, return colors, and overall structure. Why can a simple letter-by-letter check give incorrect feedback when a letter appears more than once?"

Purpose: Fix duplicate-letter scoring while preserving the expected feedback behavior.

4. Task 2 — game flow

User prompt: "Move on to Task 2 in game.py. Make sure the game allows six valid guesses, invalid inputs do not use a turn, a correct guess wins immediately, a loss reveals the target word, and entering q quits."

Purpose: Implement the required game flow and edge-case behavior.

5. Task 3 — word-length modes

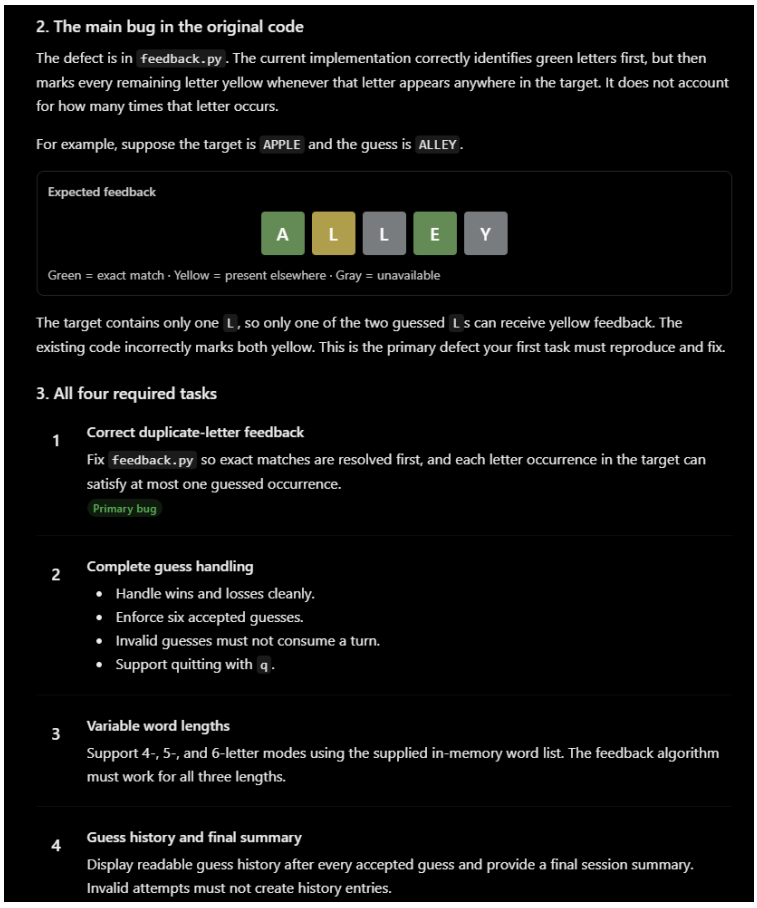
User prompt: "Proceed to Task 3. Add selectable four-, five-, and six-letter game modes using main.py and words.py, without breaking the earlier fixes. Why is it useful to keep word selection and game logic separated?"

Purpose: Add configurable word lengths while preserving Tasks 1 and 2.

6. Task 4 — guess history and summary

User prompt: "Now implement Task 4. Show the history of accepted guesses after each turn, and display a final summary for wins, losses, and quits. Include the target word, number of valid guesses, and outcome."

Purpose: Add the requested history and end-of-game summary



2. The main bug in the original code

The defect is in `feedback.py`. The current implementation correctly identifies green letters first, but then marks every remaining letter yellow whenever that letter appears anywhere in the target. It does not account for how many times that letter occurs.

For example, suppose the target is `APPLE` and the guess is `ALLEY`.

Expected feedback

A L L E Y

Green = exact match · Yellow = present elsewhere · Gray = unavailable

The target contains only one `L`, so only one of the two guessed `L`s can receive yellow feedback. The existing code incorrectly marks both yellow. This is the primary defect your first task must reproduce and fix.

3. All four required tasks

1 **Correct duplicate-letter feedback**

Fix `feedback.py` so exact matches are resolved first, and each letter occurrence in the target can satisfy at most one guessed occurrence.

Primary bug

2 **Complete guess handling**

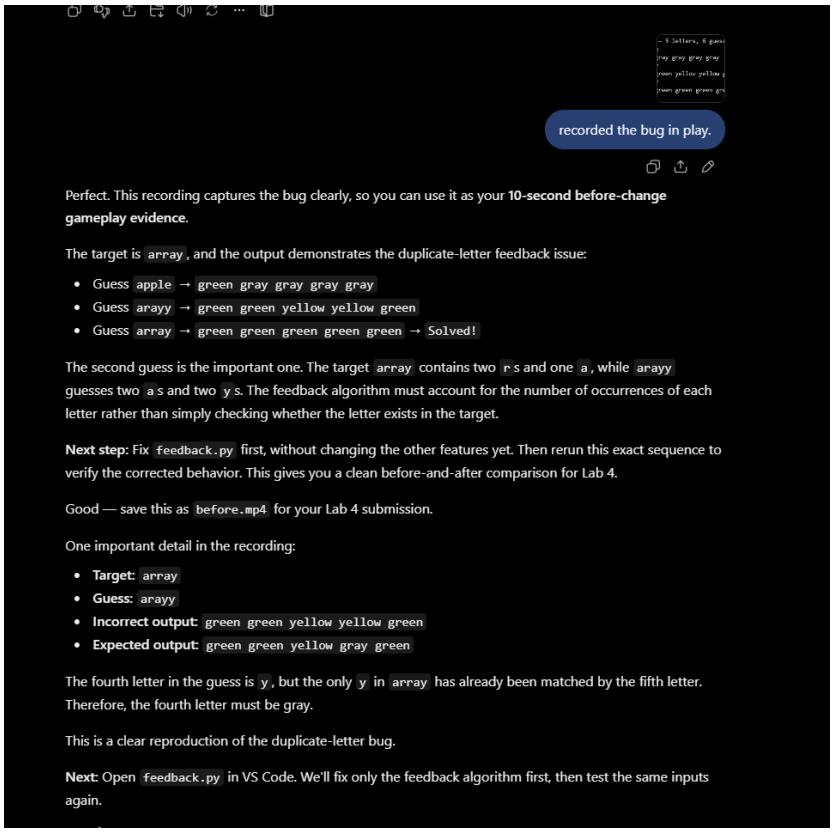
- Handle wins and losses cleanly.
- Enforce six accepted guesses.
- Invalid guesses must not consume a turn.
- Support quitting with `q`.

3 **Variable word lengths**

Support 4-, 5-, and 6-letter modes using the supplied in-memory word list. The feedback algorithm must work for all three lengths.

4 **Guess history and final summary**

Display readable guess history after every accepted guess and provide a final session summary. Invalid attempts must not create history entries.



recorded the bug in play.

Perfect. This recording captures the bug clearly, so you can use it as your **10-second before-change gameplay evidence**.

The target is `array`, and the output demonstrates the duplicate-letter feedback issue:

- Guess `apple` → `green gray gray gray gray`
- Guess `arayy` → `green green yellow yellow green`
- Guess `array` → `green green green green green` → `Solved!`

The second guess is the important one. The target `array` contains two `r`s and one `a`, while `arayy` guesses two `a`s and two `y`s. The feedback algorithm must account for the number of occurrences of each letter rather than simply checking whether the letter exists in the target.

Next step: Fix `feedback.py` first, without changing the other features yet. Then rerun this exact sequence to verify the corrected behavior. This gives you a clean before-and-after comparison for Lab 4.

Good — save this as `before.mp4` for your Lab 4 submission.

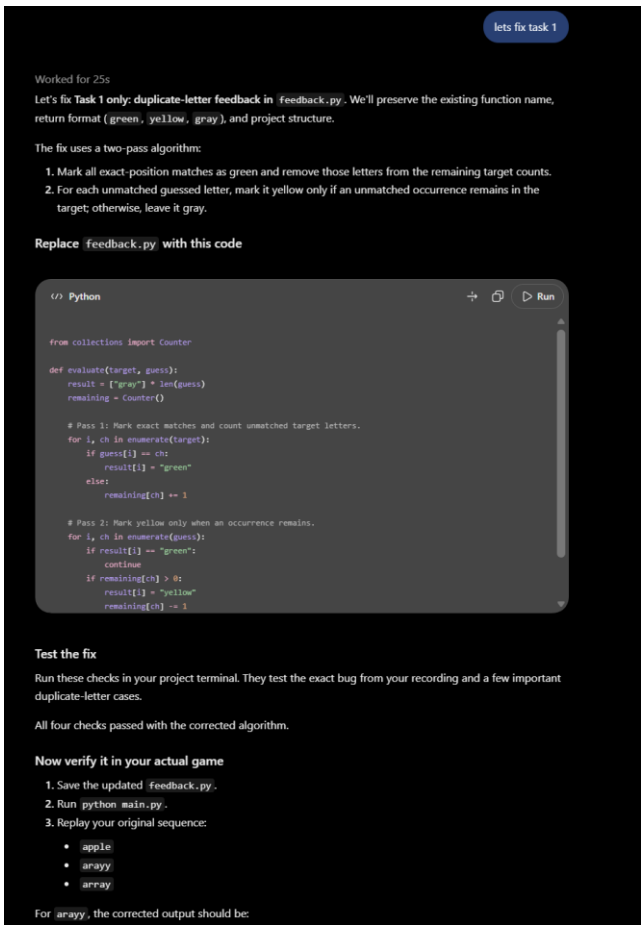
One important detail in the recording:

- **Target:** `array`
- **Guess:** `arayy`
- **Incorrect output:** `green green yellow yellow green`
- **Expected output:** `green green yellow gray green`

The fourth letter in the guess is `y`, but the only `y` in `array` has already been matched by the fifth letter. Therefore, the fourth letter must be gray.

This is a clear reproduction of the duplicate-letter bug.

Next: Open `feedback.py` in VS Code. We'll fix only the feedback algorithm first, then test the same inputs again.



lets fix task 1

Worked for 25s

Let's fix **Task 1 only: duplicate-letter feedback in** `feedback.py`. We'll preserve the existing function name, return format (`green`, `yellow`, `gray`), and project structure.

The fix uses a two-pass algorithm:

1. Mark all exact-position matches as green and remove those letters from the remaining target counts.
2. For each unmatched guessed letter, mark it yellow only if an unmatched occurrence remains in the target; otherwise, leave it gray.

Replace `feedback.py` with this code

Python Run

```python
from collections import Counter

def evaluate(target, guess):
    result = ["gray"] * len(guess)
    remaining = Counter()

    # Pass 1: Mark exact matches and count unmatched target letters.
    for i, ch in enumerate(target):
        if guess[i] == ch:
            result[i] = "green"
        else:
            remaining[ch] += 1

    # Pass 2: Mark yellow only when an occurrence remains.
    for i, ch in enumerate(guess):
        if result[i] == "green":
            continue
        if remaining[ch] > 0:
            result[i] = "yellow"
            remaining[ch] -= 1
```

Test the fix

Run these checks in your project terminal. They test the exact bug from your recording and a few important duplicate-letter cases.

All four checks passed with the corrected algorithm.

Now verify it in your actual game

1. Save the updated `feedback.py`.
2. Run `python main.py`.
3. Replay your original sequence:
   - `apple`
   - `arayy`
   - `array`

For `arayy`, the corrected output should be:

Postfix

```
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab4\17_wordle> python main.py
Choose word length (4, 5, or 6): 4
Wordle - 4 letters, 6 guesses.
> gaem
gray gray yellow gray

Guess History:
1. gaem - gray gray yellow gray

> coed
gray gray yellow gray

Guess History:
1. gaem - gray gray yellow gray
2. coed - gray gray yellow gray

> bluu
green green green gray

Guess History:
1. gaem - gray gray yellow gray
2. coed - gray gray yellow gray
3. bluu - green green green gray

> lbbu
yellow yellow gray yellow

Guess History:
1. gaem - gray gray yellow gray
2. coed - gray gray yellow gray
3. bluu - green green green gray
4. lbbu - yellow yellow gray yellow

> lbbb
yellow yellow gray gray

Guess History:
1. gaem - gray gray yellow gray
2. coed - gray gray yellow gray
3. bluu - green green green gray
4. lbbu - yellow yellow gray yellow
5. lbbb - yellow yellow gray gray

> blueeee
Enter a valid 4-letter word.
```

```
> blue
green green green green

Guess History:
1. gaem - gray gray yellow gray
2. coed - gray gray yellow gray
3. bluu - green green green gray
4. lbbu - yellow yellow gray yellow
5. lbbb - yellow yellow gray gray
6. blue - green green green green

Solved!

--- Session Summary ---
Outcome: Won
Target word: blue
Valid guesses: 6/6
Total guesses:
1. gaem - gray gray yellow gray
2. coed - gray gray yellow gray
3. bluu - green green green gray
4. lbbu - yellow yellow gray yellow
5. lbbb - yellow yellow gray gray
6. blue - green green green green
-----------------------
PS C:\Vedant\PESU\Semester5\SE\Lab\Lab4\17_wordle>
```